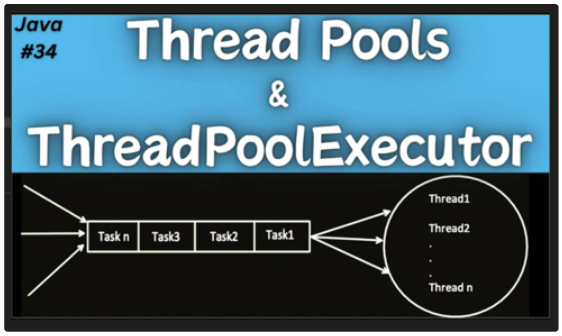
Distributed Scheduler - Part1 (Scheduler Fundamental)

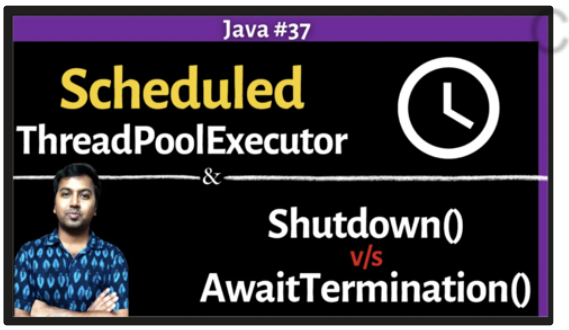
What is Scheduler?

It is just a background thread that sleeps until a given time, wakes up, runs a method, then sleeps again.

Springboot does no magic, internally is just uses java: **ScheduledThreadPoolExecutor**

Also discussed in depth in Java series:





Concept && Coding

let first see what happens internally? Before we cover how Springboot abstracted all of this for us

Example to Schedule a task:

ScheduledThreadPoolExecutor executor = new ScheduledThreadPoolExecutor(3);

Runnable task = () → System.out.println("Hello Shrayansh!");

executor.schedule(task, 5, TimeUnit.SECONDS);

Few ScheduledExecutorService Methods:

S.No.	Method Signature	Description
1.	schedule(Runnable command, long delay, TimeUnit unit)	• Runs only once. • Delay is counted from Task submission time example: executor.schedule(task, 5, TimeUnit.SECONDS); Time(Sec) : 0 1 2 3 4 5-Task started 6
2.	scheduleAtFixedRate(Runnable command, long initialDelay, long period, TimeUnit unit)	• Run the Task repeatedly at a fixed Rate. • Concept && Coding example: executor.scheduleAtFixedRate(task, 2, 4, TimeUnit.SECONDS); Time(Sec) : 0 1 2- Task started 3 4 5 6- Task started 7 8 9 10- Task started

11

initial delay = 2 sec :

2sec: Task started

task finished → requeued again with next run time: 2(previous start time)+4(delay)= 6sec

6sec: Task started

Task finished → requeued again with next run time: 6+4sec = 10sec

10sec : Task started

Task finished → requeued again with next run time: 10+4sec = 14sec

Concept && Coding

14sec : Task started

.

And so on...

.

<u>Pls note (Important):</u>

if interviewer ask, what if Task is taking too much time, what would happen in that case?

<u>Answer is:</u>

-

Even if Task takes too much time, it always run in sequential manner. Why because, same Task (object) is requeued. Lets understand this with an example:

		executor.scheduleAtFixedRate(task, 2, 4, TimeUnit.SECONDS); initial delay = 2 sec : 2sec: Task started 12sec: Task finished. Now it need to compute the new time when this task need to be start again. new time = previous start time + fixed delay time new time = 2sec + 4sec = 6sec But 6sec < current time (12sec) new time = 6sec + 4sec = 10sec but 10sec < current time (12sec) new time = 10sec + 4sec = 14sec but 14sec >= current time (12sec) So requeued this task again with this time 14sec.
3.	scheduleWithFixedDelay(Runnable command, long initialDelay, long delay, TimeUnit unit)	• Run the Task repeatedly with fixed delay. • new time is computed : New time = task when finished + Delay example: • executor.scheduleWithFixedDelay(task, 2, 4, TimeUnit.SECONDS); initial delay = 2 sec :

Concept && Coding

		2sec: Task started 12sec: task finished New run time is computed : previous finished time + delay 12sec + 4sec = 16sec 16sec: Task started 18sec: Task finished New run time is computed : previous finished time + delay. 18sec+4sec = 22sec 22sec: Task started . and so on... .

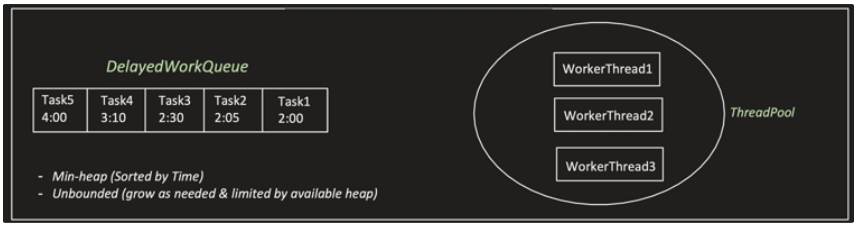
Concept && Coding

So, when we did:

ScheduledThreadPoolExecutor executor = new ScheduledThreadPoolExecutor(3);

We created scheduled thread pool with 3 worker threads.

ScheduledThreadPoolExecutor



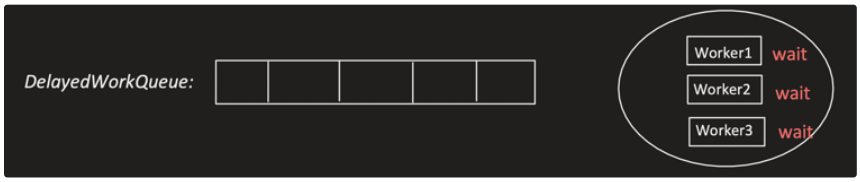
executor.schedule(task, 5, TimeUnit.SECONDS);

- Internally it creates a ScheduleTask (*which has RunnableTask and delay time information*)
- And insert it into the DelayedWorkQueue

Lets see the flow:

Not Optimized:

Step1: DelayedWorkQueue is empty and all WorkerThreads are in wait state.



Note: Threads are created in Runnable state but they look for task in Queue, since there is no task, they move to wait state.

```
public RunnableScheduledFuture<?> take() throws InterruptedException {
    final ReentrantLock lock = this.lock;
    lock.lockInterruptibly();
    try {
        for (;;) {
          /*
          Looks at the head of the queue, if there is no task.
          goes to the wait state.
           */
            RunnableScheduledFuture<?> first = queue[0];
            if (first == null)
                available.await();
            else {
                  . . .
            }
        }
    } finally {
         . . .
    }
}

```

Concept && Coding

Step2:

- First Task arrives in the delayed queue.
- It invokes only 1 worker thread which are in waiting state.

```
public boolean offer(Runnable x) {
    if (x == null)
        throw new NullPointerException();
    RunnableScheduledFuture<?> e = (RunnableScheduledFuture<?>) x;
    final ReentrantLock lock = this.lock;
    lock.lock();
    try {
```

```java
        int i = size;
        if (i >= queue.length)
          /*
          Increase the size of queue(by 50%) if its full.
           */
            grow();
        size = i + 1;
        if (i == 0) {
            queue[0] = e;
            setIndex(e, 0);
        } else {
          /*
          Heapify after insertion
          */
            siftUp(i, e);
        }
        if (queue[0] == e) {
            leader = null;
          /*
          If this task is the only item, means all threads might be in
          waiting state.
          So randomly invokes/notify 1 thread which is in wait state.
          */
            available.signal();
        }
    } finally {
        lock.unlock();
    }
    return true;
}

```

Step3:

Prompt && Coding

- Worker1 checks the head task.

  Two possibilities:

Possibility 1:

- Current time >= Task1 Run time
- It will take out the task from the queue and start working on it.

  Lets say: current time is 2:01

Possibility 2:

- Current time < Task1 Run time

  - Compute delay: Task1 Run time - Current time = X seconds
  - Goes into **Timed Waiting** state.
    - JVM parks the thread (does not ***busy-wait***)
    - Means it will be in wait state for only X seconds after that it will get awake(till then it do not consume any CPU cycles)
      - It make use of OS level timers, so when the timer expires, OS wakes the thread and control return to JVM.

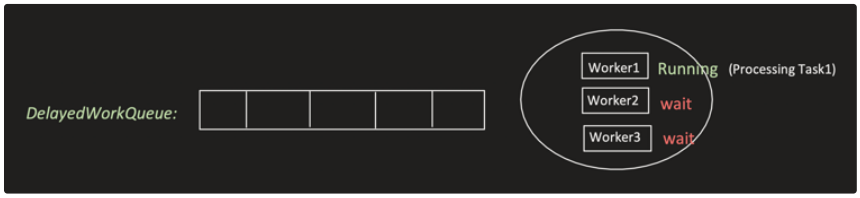
Let say current time is: 1:55PM

Delay remaining = 2PM - 1:55PM = 5minutes

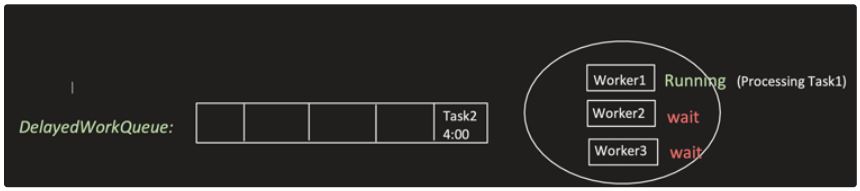
```java
public RunnableScheduledFuture<?> take() throws InterruptedException {
    final ReentrantLock lock = this.lock;
    lock.lockInterruptibly();
    try {
        for (;;) {
            RunnableScheduledFuture<?> first = queue[0];
            if (first == null)
                available.await();
            else {
                long delay = first.getDelay(NANOSECONDS);
                if (delay <= 0L)
                  /*
                  Possibility1 scenario:
                  If delay is <=0 means its ready to be picked now.
                  Poll/remove from the Delayed queue.
                  */
                    return finishPoll(first);
                first = null; // don't retain ref while waiting
                if (leader != null)
                    available.await();
                else {
                    Thread thisThread = Thread.currentThread();
                    leader = thisThread;
                    try {
                      /*
                      Possibility2 scenario:
                      If task is not yet ready for execution,
                      Go into Timed Waiting state.
                      */
                        available.awaitNanos(delay);
                    } finally {
                        if (leader == thisThread)
                            leader = null;
                    }
                }
            }
        }
    } finally {
        if (leader == null && queue[0] != null)
            available.signal();
        lock.unlock();
    }
}

```

Concept && Coding

- After Timed Wait state is over, Worker1 thread wakes up.
- Check the queue again.
- And if Task run time is expired. Remove from the queue and process it.



- Now, 1 thread which was awake, is now going to process the Task.
- But if it process the task, then no one will be looking at the Delayed queue.
- So, if there is more task in the delayed queue, it invokes/notify 1 more thread which is in waiting state.




```java
public RunnableScheduledFuture<?> take() throws InterruptedException {
    final ReentrantLock lock = this.lock;
    lock.lockInterruptibly();
    try {
        for (;;) {
            RunnableScheduledFuture<?> first = queue[0];
            if (first == null)
                available.await();
            else {
                long delay = first.getDelay(NANOSECONDS);
                if (delay <= 0L)
                  /*
                  Active thread, say picks the task which is ready and start
                  executing it.
                  */
                    return finishPoll(first);
                first = null; // don't retain ref while waiting
                if (leader != null)
                    available.await();
                else {
                    Thread thisThread = Thread.currentThread();
                    leader = thisThread;
                    try {
                        available.awaitNanos(delay);
                    } finally {
                        if (leader == thisThread)
                            leader = null;
                    }
                }
            }
        }
    } finally {
      /*
      In finally block, it checks if there is more task in the queue,
      send the signal/notify to wakes up 1 more thread.
      */
        if (leader == null && queue[0] != null)
            available.signal();
        lock.unlock();
    }
}

```

Concept && Coding

And process keeps on repeating.....

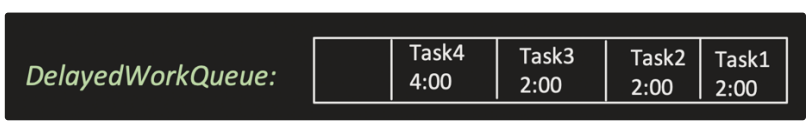
```java
public RunnableScheduledFuture<?> take() throws InterruptedException {
    final ReentrantLock lock = this.lock;
    lock.lockInterruptibly();
    try {
      /*
      Infinite loop
      */
        for (;;) {
            RunnableScheduledFuture<?> first = queue[0];
            if (first == null)
                available.await();
            else {
                long delay = first.getDelay(NANOSECONDS);
                if (delay <= 0L)
                    return finishPoll(first);
                first = null; // don't retain ref while waiting
                if (leader != null)
                    available.await();
                else {
                    Thread thisThread = Thread.currentThread();
                    leader = thisThread;
                    try {
                        available.awaitNanos(delay);
                    } finally {
                        if (leader == thisThread)
                            leader = null;
                    }
                }
            }
        }
    } finally {
        if (leader == null && queue[0] != null)
            available.signal();
        lock.unlock();
    }
}

```

Concept && Coding

<u>But why I mentioned, its not Optimized:</u>

Lets see the below use case:

Current time: 2:00



Thread1 : Runnable state

- Check the head of the Queue, task1 is ready to be picked.
- So it remove the Task1 and before it going to process it. It also invokes 1 more waiting thread to look at the queue.

Thread1 : Running state

Thread2 : Runnable state

- Check the head of the Queue, task2 is ready to be picked.
- So it remove the Task2 and before it going to process it. It also invokes 1 more waiting thread to look at the queue.

Thread2 : Running state

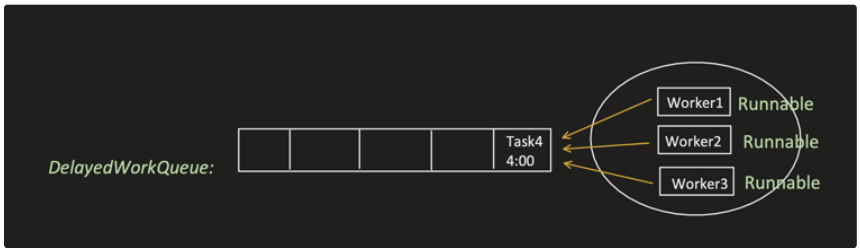
Thread3 : Runnable state

- Check the head of the Queue, task3 is ready to be picked.
- So it remove the Task3 and before it going to process it. It also invokes 1 more waiting thread to look at the queue. But there are no more thread in the queue, all are busy.

Thread3 : Running state

Concept && Coding

At 3:00 - All three worker threads finished their task



Now, all 3 will compete for the Task present in Delayed queue.

Say:

Thread1: Acquired Lock first

- Checks the Run time is not yet expired
- Computes delay time

- Goes into Timed_Waiting state

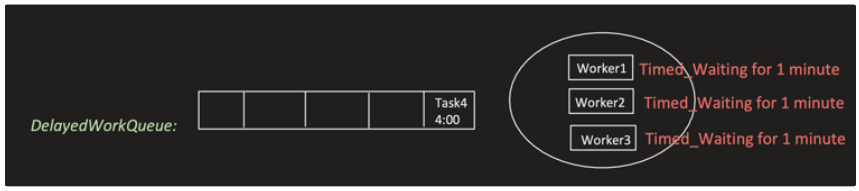
Thread2: Acquired Lock

- Checks the Run time is not yet expired
- Computes delay time
- Goes into Timed_Waiting state

Thread3: Acquired Lock

- Checks the Run time is not yet expired
- Computes delay time
- Goes into Timed_Waiting state

Concept && Coding



So, if you see. Unnecessarily all 3 goes into Timed waiting.

So, what's the efficient solution?

Its optimization is:

- Only 1 thread should go into Timed Waiting.
- Others goes into waiting (indefinite) waiting state, till someone not wake them up.

Why unnecessarily OS maintain the timer for each thread.

That's where : LEADER - FOLLOWER optimization comes into picture.

Very basic idea is:

- Only 1 thread should go in TIMED_WAITING.
- Others should just go into waiting (indefinitely) state.
- And there is no specific leader thread or follower thread, its all managed through just 1 variable.

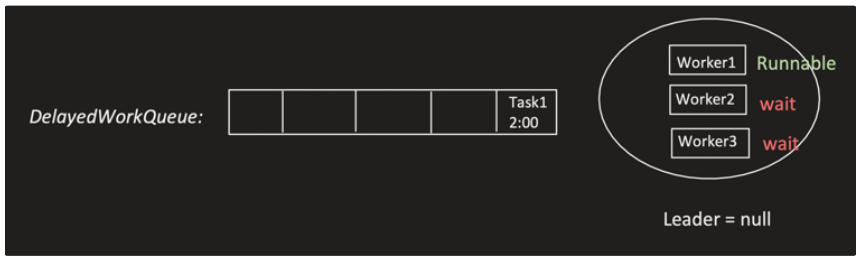
<u>Step1 :</u> No change

DelayedWorkQueue is empty and all WorkerThreads are in wait state.

Concept && Coding

<u>Step2 :</u> No change

- First Task arrives in the delayed queue.
- It invokes only 1 worker thread which are in waiting state.



<u>Step3 :</u>

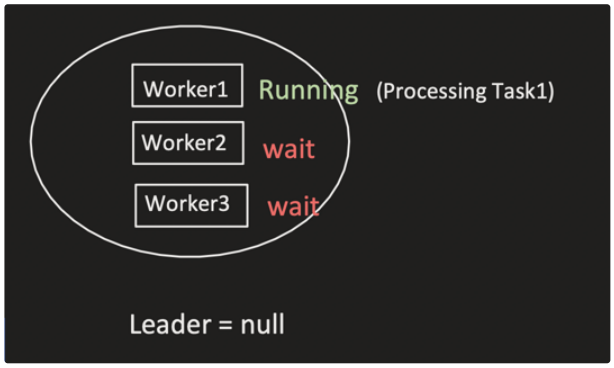
- Worker1 checks the head task.

Two possibilities:

<u>Possibility 1</u>: No change

- Current time >= Task1 Run time
- It will take out the task from the queue and start working on it.

  Lets say: current time is 2:01




```java
public RunnableScheduledFuture<?> take() throws InterruptedException {
    final ReentrantLock lock = this.lock;
    lock.lockInterruptibly();
    try {
        for (;;) {
            RunnableScheduledFuture<?> first = queue[0];
            if (first == null)
                available.await();
            else {
                long delay = first.getDelay(NANOSECONDS);
                if (delay <= 0L)
                  /*
                  Possibility1 scenario:
                  If delay is <=0 means its ready to be picked now.
                  Poll/remove from the Delayed queue.
                  */
                    return finishPoll(first);
                first = null; // don't retain ref while waiting
                if (leader != null)
                    available.await();
                else {
                    Thread thisThread = Thread.currentThread();
                    leader = thisThread;
                    try {
                        available.awaitNanos(delay);
                    } finally {
                        if (leader == thisThread)
                            leader = null;
                    }
                }
            }
        }
    } finally {
      /*
      Leader == null, Yes
      More items in the queue, notify 1 thread which is in waiting state.
      */
        if (leader == null && queue[0] != null)
            available.signal();
        lock.unlock();
    }
}

```

Concept && Coding

Possibility 2: ***This is where Leader variable is used***

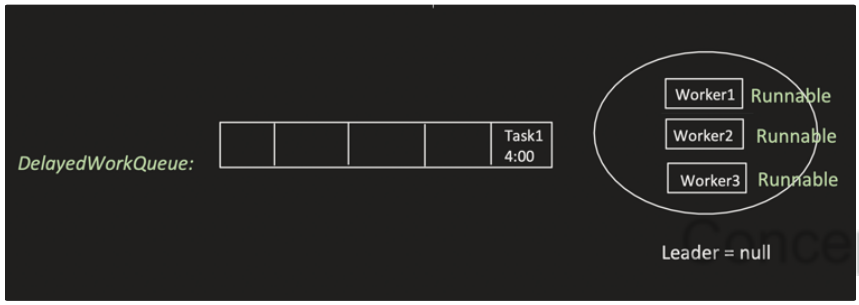
Because at this point, thread will go in TIMED_WAIT state and we need to make sure that only 1 thread should go in TIMED_WAIT state, not more than that.
Others apart from leader should go in Wait (indefinite) state.

- Current time < Task1 Run time
  - Compute delay: Task1 Run time - Current time = X seconds
  - If **LEADER** variable is not null *(mean there is already some leader present)*
    - Goes into WAIT state.

Else

Make itself LEADER and GOES into **Timed Waiting** state for X seconds

- When Timer is up and it get wake up by OS, it set back LEADER = null



Concept && Coding

Current time is : 3:00

Thread1: Acquired Lock first

- Checks the Run time is not yet expired
- Computes delay time = x second
- Leader = null
- Set, Leader = Thread.currentThread();
- Release the lock + goes into timed waiting state (awaitNanos(x))

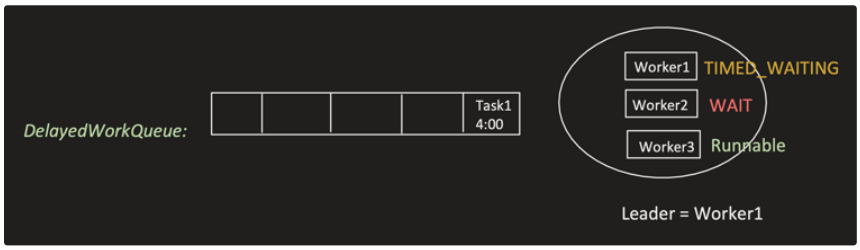
```
public RunnableScheduledFuture<?> take() throws InterruptedException {
    final ReentrantLock lock = this.lock;
```

```java
    lock.lockInterruptibly();
    try {
        for (;;) {
            RunnableScheduledFuture<?> first = queue[0];
            if (first == null)
                available.await();
            else {
                long delay = first.getDelay(NANOSECONDS);
                if (delay <= 0L)
                    return finishPoll(first);
                first = null; // don't retain ref while waiting
                if (leader != null)
                    available.await();
                else {
                  /*
                  - Leader = null
                  - Set, Leader = Thread.currentThread();
                - Release the lock + goes into timed waiting state (awaitNanos(x))
                  */
                    Thread thisThread = Thread.currentThread();
                    leader = thisThread;
                    try {
                        available.awaitNanos(delay);
                    } finally {
                        if (leader == thisThread)
                            leader = null;
                    }
                }
            }
        }
    } finally {
        if (leader == null && queue[0] != null)
            available.signal();
        lock.unlock();
    }
}

```

Concept && Coding

Thread2: Acquired Lock

- Checks the Run time is not yet expired
- Computes delay time = x second
- Leader != null
- Release the lock + goes into waiting state (indefinitely)

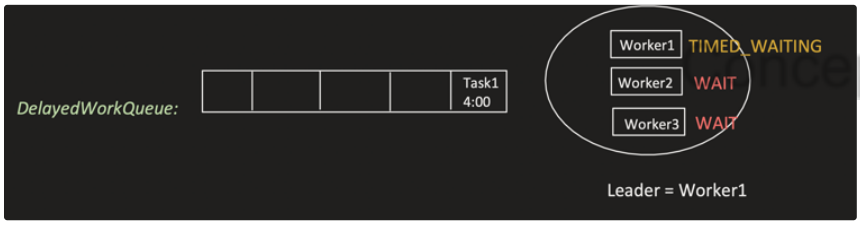



```java
public RunnableScheduledFuture<?> take() throws InterruptedException {
    final ReentrantLock lock = this.lock;
    lock.lockInterruptibly();
    try {
        for (;;) {
            RunnableScheduledFuture<?> first = queue[0];
            if (first == null)
                available.await();
            else {
                long delay = first.getDelay(NANOSECONDS);
                if (delay <= 0L)
                    return finishPoll(first);
                first = null; // don't retain ref while waiting
                if (leader != null)
                  /*
                  - Leader != null
```

```
                    Release the lock + goes into waiting state (indefinitely)
                    */
                    available.await();
                else {
                    Thread thisThread = Thread.currentThread();
                    leader = thisThread;
                    try {
                        available.awaitNanos(delay);
                    } finally {
                        if (leader == thisThread)
                            leader = null;
                    }
                }
            }
        }
    } finally {
        if (leader == null && queue[0] != null)
            available.signal();
        lock.unlock();
    }
}

```

Thread3: Acquired Lock

- Checks the Run time is not yet expired
- Computes delay time = x second
- Leader != null
- Release the lock + goes into waiting state (indefinitely)

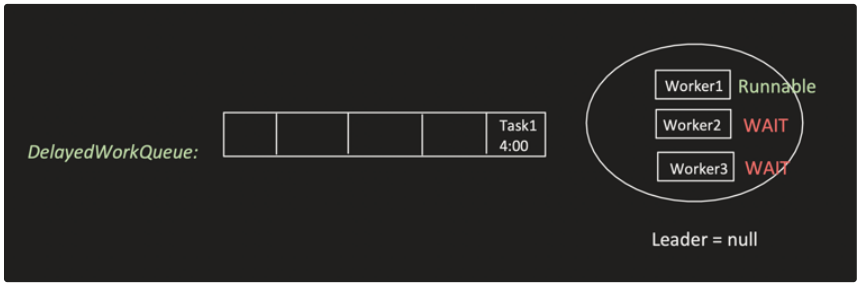


Concept && Coding

Now, its 4:00 → WorkerThread1 wakes up

- Set Leader = null

- And Step3 process repeats




```
public RunnableScheduledFuture<?> take() throws InterruptedException {
    final ReentrantLock lock = this.lock;
    lock.lockInterruptibly();
    try {
```

```java
        for (;;) {
            RunnableScheduledFuture<?> first = queue[0];
            if (first == null)
                available.await();
            else {
                long delay = first.getDelay(NANOSECONDS);
                if (delay <= 0L)
                    return finishPoll(first);
                first = null; // don't retain ref while waiting
                if (leader != null)
                    available.await();
                else {
                    Thread thisThread = Thread.currentThread();
                    leader = thisThread;
                    try {
                        available.awaitNanos(delay);
                    } finally {
                      /*
                      WorkerThread1  wakes up
                      - Set Leader = null
                      - And repeat the process
                      */
                        if (leader == thisThread)
                            leader = null;
                    }
                }
            }
        }
    } finally {
        if (leader == null && queue[0] != null)
            available.signal();
        lock.unlock();
    }
}

```

Concept && Coding